OMNILEGAL AI: DOMAIN-SPECIFIC LEGAL RESEARCH ASSISTANT USING HYBRID RAG, MULTI-MODEL SYNTHESIS AND CITATION EVALUATION

Submitted by

[STUDENT NAME 1] - [REGISTER NUMBER 1]

[STUDENT NAME 2] - [REGISTER NUMBER 2]

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

RAJALAKSHMI ENGINEERING COLLEGE (AUTONOMOUS), THANDALAM

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ANNA UNIVERSITY, CHENNAI 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this thesis titled OmniLegal AI: Domain-Specific Legal Research Assistant Using Hybrid RAG, Multi-Model Synthesis and Citation Evaluation is the bonafide work of [STUDENT NAME 1] ([REGISTER NUMBER 1]) and [STUDENT NAME 2] ([REGISTER NUMBER 2]), who carried out the project work under my supervision.

Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation based on which a degree or award was conferred on an earlier occasion on these or any other candidates.

Dr. J M GNANASEKAR

Head of the Department

Department of Artificial Intelligence and Data Science

Rajalakshmi Engineering College, Thandalam, Chennai - 602105.

[GUIDE NAME]

[Guide Designation]

Department of Artificial Intelligence and Data Science

Rajalakshmi Engineering College, Thandalam, Chennai - 602105.

Certified that the candidates were examined in the VIVA-VOCE Examination held on [DATE].

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the thesis titled OmniLegal AI: Domain-Specific Legal Research Assistant Using Hybrid RAG, Multi-Model Synthesis and Citation Evaluation is a bonafide work carried out by us under the supervision of [GUIDE NAME], [Guide Designation], Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, Thandalam, Chennai.

[STUDENT NAME 1] ([REGISTER NUMBER 1])

[STUDENT NAME 2] ([REGISTER NUMBER 2])

ACKNOWLEDGEMENT

We thank the Almighty for giving us the strength and determination required to complete this project successfully. We sincerely thank our respected Chairman, Chairperson, Vice Chairman, Principal, Head of the Department, and the Department of Artificial Intelligence and Data Science for providing the required infrastructure and academic support throughout this work.

We express our sincere gratitude to [GUIDE NAME], [Guide Designation], Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, for valuable guidance, encouragement, technical suggestions, and continuous support during the development of this project.

We also thank our project coordinator, faculty members, friends, and family members for their support and encouragement. Their feedback helped us improve the design, implementation, testing, and documentation of OmniLegal AI.

ABSTRACT

Legal research requires the interpretation of complex statutes, treaties, judicial decisions, and scholarly commentary across multiple jurisdictions. Manual legal research is time-consuming and requires careful source verification, especially when a question involves international law, Indian constitutional law, comparative law, or Model United Nations policy preparation. Recent progress in Large Language Models (LLMs) has made automated legal assistance possible, but general-purpose chat systems often produce unsupported claims, weak citations, and inconsistent reasoning. This project presents OmniLegal AI, a domain-specific legal research assistant designed to answer legal questions using retrieval-augmented generation, hybrid search, structured legal analysis, multi-model deliberation, and citation verification.

The system is implemented in Python and provides both Streamlit and Chainlit interfaces. The backend is built around a shared LangGraph pipeline that classifies user input, extracts legal entities and issues, retrieves relevant authorities from indexed legal corpora, performs jurisdiction-specific legal analysis, synthesizes a final answer, and verifies citations before presenting the output. The system supports multiple workflows, including legal question answering, legal entity analysis, international-versus-Indian conflict discovery, Indian stance prediction, one-page MUN brief generation, debate preparation, corpus exploration, and local benchmark execution.

The project integrates legal corpora such as the Indian Constitution, UN Charter, ICCPR, ICESCR, Malcolm Shaw's international law textbook, remote case-law sources, curated dataset registries, and donor-inspired legal NLP patterns. Retrieval is supported through Qdrant or a SQLite fallback, dense embeddings using BAAI/bge-m3, sparse retrieval, RRF fusion, and optional BAAI/bge-reranker-v2-m3 reranking. The generation layer uses Groq-hosted Llama 3.3 70B as the primary LLM, with optional contextual retrieval support from Gemini 2.5 Flash and local model fallbacks.

Evaluation is performed using local smoke benchmarks, citation correctness checks, retrieval recall, source diversity, macro-F1 for conflict detection, argument span coverage, RAGAS-based faithfulness gates, and LegalBench-style legal evaluation runs. Existing artifacts show strong citation existence and quote-match performance in smoke tests, while production completion gates identify areas requiring further improvement, especially RAGAS faithfulness and ingestion metadata quality. Overall, OmniLegal AI demonstrates how domain-specific RAG and structured verification can improve the reliability of legal research assistants while highlighting the need for larger gold datasets, stronger evaluation, and careful legal-disclaimer handling.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

Artificial Intelligence has transformed the way domain-specific knowledge systems are designed. In the legal domain, the need for accurate information retrieval and source-grounded reasoning is especially important because a small error in statutory interpretation, case citation, or jurisdictional context can change the meaning of an answer. Legal users often work with long documents such as treaties, constitutional provisions, judicial opinions, international law commentaries, and policy materials. These documents are dense, hierarchical, and citation-heavy. Traditional keyword search can retrieve relevant documents, but it does not automatically explain their meaning, compare legal positions, or prepare a structured answer for debate and research use.

Large Language Models can generate fluent legal explanations, but standalone LLMs are not sufficient for serious legal research because they may hallucinate citations, rely on outdated memory, or miss jurisdiction-specific distinctions. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding model responses in retrieved source passages. A legal RAG system must go beyond ordinary document search. It should support jurisdiction routing, issue classification, legal entity extraction, citation verification, source diversity, and evaluation against legal benchmark tasks.

OmniLegal AI is designed as a legal research assistant for international law, Indian law, comparative legal analysis, and Model United Nations research workflows. The system combines a curated legal corpus, remote source ingestion, hybrid retrieval, typed schemas, LLM-based synthesis, deterministic citation verification, benchmark tracking, and multiple user interfaces. It is not a replacement for legal advice, but it is a structured research tool that helps users locate authorities, understand issues, compare jurisdictions, and prepare cited legal arguments.

1.2 Motivation

The motivation for this project arises from the difficulty of conducting accurate legal research under time constraints. Students, researchers, and MUN delegates often need quick answers to questions involving international law, human rights, armed conflict, treaty interpretation, Indian constitutional law, and diplomatic policy. A user may ask whether anticipatory self-defense is lawful under Article 51 of the UN Charter, how Indian domestic law interacts with ICCPR obligations, or how a delegate should frame a debate position. Answering such questions requires more than a short summary. It requires relevant authorities, jurisdiction-specific reasoning, comparison of legal positions, and a clear explanation of uncertainty.

General-purpose chatbots can provide broad explanations, but they may not reliably show the exact source passages used. Legal research platforms provide databases and search tools, but they often require manual review and may be expensive or inaccessible for academic projects. OmniLegal AI is motivated by the need for an accessible, modular, and evaluation-aware legal assistant that can combine legal corpora with LLM reasoning while preserving source grounding.

Another motivation is the need to support academic evaluation. Many generative AI projects stop at producing answers, but do not measure whether the output is faithful to the source material. OmniLegal AI includes benchmark artifacts, smoke tests, citation existence checks, quote matching, retrieval recall, macro-F1 for classification-style tasks, RAGAS gates, and production completion reports. This makes the system more suitable for a project report because it can be described, tested, and improved through measurable criteria.

1.3 Problem Statement

Legal materials are complex, lengthy, and jurisdiction-sensitive. A single legal question may require interpreting treaty provisions, domestic constitutional rules, case-law principles, and academic commentary. Users without legal expertise may struggle to identify the relevant documents, understand the hierarchy of authorities, and distinguish supported legal conclusions from unsupported model output. Existing systems either provide raw search results without synthesis, or provide generated answers without enough verifiable grounding.

The problem addressed in this project is the development of a domain-specific legal research assistant that can accept natural language legal questions, identify the legal issue and jurisdiction, retrieve relevant legal authorities, generate a structured answer, and verify citations before presenting the response. The system must also support related workflows such as legal entity intake, international-versus-domestic conflict discovery, Indian stance prediction, MUN brief generation, debate coaching, corpus exploration, and benchmark evaluation.

The project also addresses the challenge of evaluating generative legal AI. Since legal answers may be paraphrased, exact text-overlap metrics alone are insufficient. OmniLegal therefore combines retrieval metrics, citation correctness, quote matching, smoke benchmarks, RAGAS faithfulness, LegalBench-style evaluation, and manual gold files for selected tasks.

1.4 Objectives

The primary objective of OmniLegal AI is to design and implement a legal research assistant that produces citation-grounded legal answers using a modular and benchmarkable architecture. The system aims to:

1. Build a domain-specific legal RAG pipeline for international law, Indian law, and comparative legal analysis.
2. Ingest and index legal corpora, including treaties, constitutional texts, case law, commentary, and curated legal NLP datasets.
3. Classify user questions into legal issue categories such as use of force, human rights, treaty interpretation, state responsibility, jurisdiction and immunity, and general international law.
4. Extract legal entities, named cases, jurisdictions, temporal references, and issue labels from user input.
5. Retrieve relevant passages using hybrid retrieval, dense embeddings, collection routing, source diversity, and reranking.
6. Generate structured jurisdiction-specific legal analysis using IRAC-style reasoning.
7. Synthesize final answers with legal citations and identify conflicts between international and domestic legal positions.
8. Verify citations using marker checks, quote matching, lexical support, optional NLI verification, and fallback grounded drafts.
9. Provide multiple user interfaces through Streamlit pages and a Chainlit chat workflow.
10. Support MUN-specific workflows including Indian stance prediction, one-page brief generation, debate cards, likely POIs, rebuttals, and negotiation red lines.
11. Evaluate the system using retrieval recall, citation coverage, quote matching, macro-F1, argument coverage, RAGAS faithfulness, LegalBench-style runs, and smoke tests.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Systems

Legal technology systems can be broadly divided into traditional search platforms, legal NLP benchmark systems, general-purpose LLM assistants, and retrieval-augmented legal AI systems.

Traditional legal research platforms such as commercial legal databases provide access to statutes, cases, commentary, and citation networks. These platforms are powerful for professional legal research, but they often require subscription access and depend heavily on manual user interpretation. They retrieve documents but do not always produce an integrated, debate-ready answer or compare international and domestic legal positions automatically.

General-purpose LLM systems can answer legal questions in natural language and are useful for quick explanations. However, they are not always grounded in current source material and can generate incorrect or fabricated citations. Their responses may also vary between runs and may not follow a strict legal reasoning format. For academic and legal research use, this creates a reliability gap.

Legal NLP benchmark datasets such as LexGLUE, CaseHOLD, LegalBench, CLERC, LLeQA, BSARD, FairLex, and other legal retrieval or classification datasets provide important evaluation tasks. These datasets help measure legal classification, retrieval, entailment, summarization, and question answering performance. However, benchmark datasets alone do not produce a user-facing legal assistant. They must be integrated into an application pipeline with ingestion, retrieval, generation, and interface layers.

Retrieval-Augmented Generation systems combine information retrieval with LLM answer generation. In legal applications, RAG is especially useful because the answer can be grounded in retrieved legal authorities. A legal RAG system must handle long documents, citation formatting, jurisdiction filtering, document hierarchy, and evaluation of citation support. OmniLegal AI belongs to this category, but extends it by adding multi-page Streamlit workflows, Chainlit step visualization, source-adapter ingestion, model council deliberation, citation verification, and legal benchmark tracking.

2.2 Limitations of Existing Systems

Existing legal information systems have several limitations. Traditional legal databases can retrieve documents but may not generate concise explanations, debate cards, or structured MUN briefs. General-purpose chatbots can produce fluent answers, but they may hallucinate legal authorities or fail to distinguish between international, national, and comparative law. Many systems do not expose their retrieval logic, making it difficult for users to verify why a certain source was used.

Another limitation is the lack of end-to-end evaluation. Some systems report general model accuracy but do not measure whether legal citations actually exist, whether quoted text appears in the cited source, or whether the answer is faithful to the retrieved context. Legal answers require stronger verification than ordinary summarization tasks because unsupported claims can mislead users.

Many legal AI systems also lack modular extensibility. Adding new jurisdictions, new source adapters, new benchmarks, or new output formats often requires substantial redesign. OmniLegal addresses this limitation by separating configuration, data registry, ingestion, retrieval, pipeline stages, service-level workflows, frontends, and evaluation scripts.

2.3 Research Gap

The main research gap is the absence of a lightweight, academic, source-grounded legal research assistant that combines multi-jurisdiction retrieval, structured legal reasoning, MUN-oriented outputs, and built-in evaluation. Existing systems usually focus on only one part of the workflow: search, summarization, QA, classification, or benchmark evaluation.

OmniLegal AI attempts to fill this gap by providing a unified system with:

1. Multi-source legal corpus ingestion.
2. Hybrid retrieval and collection routing.
3. Legal entity and issue extraction.
4. Jurisdiction-specific legal analysis.
5. Multi-model synthesis through a model council and supreme verdict workflow.
6. Citation verification and grounded fallback behavior.
7. Dedicated MUN outputs such as briefs, stance cards, and debate coaching.
8. Benchmark and production-readiness artifacts.

The project therefore contributes not merely an LLM frontend, but a structured legal AI pipeline that can be evaluated and extended.

2.4 Base Model and System Stack Selection

OmniLegal does not depend on a single model. Instead, it uses a stack of specialized models and services selected for different parts of the legal research workflow.

For generation, the system is configured to use Groq-hosted Llama 3.3 70B Versatile through the `GROQ_LLM` setting. This model is used for high-quality legal synthesis and the Supreme Omni Model final response. For local fallback, the configuration includes qwen2.5:7b-instruct-q4_K_M through Ollama. The model council also includes a Hugging Face sequence-to-sequence expert configured as google/flan-t5-base.

For retrieval, OmniLegal uses BAAI/bge-m3 as the embedding model and BAAI/bge-reranker-v2-m3 as the reranker. The retriever combines dense vector search, sparse retrieval, RRF fusion, lexical fallback, source diversity, and collection-specific routing. Qdrant is the primary vector database, while SQLite fallback support exists for lighter local execution.

For classification and entity extraction, the configuration includes MoritzLaurer/deberta-v3-large-zeroshot-v2.0-c for zero-shot classification, en_legal_ner_trf with fallback to en_core_web_sm for spaCy NER, urchade/gliner_multi-v2.1 for flexible entity recognition, and vectara/hallucination_evaluation_model for optional NLI verification. Heavy NLP models are disabled by default through feature flags so that the system can boot in a lightweight heuristic mode.

For contextual retrieval and chunk enhancement, the code supports Gemini 2.5 Flash as a contextual summarization provider. This is used to generate document-level context strings that can improve retrieval quality.

This layered model selection is appropriate because legal research requires different capabilities at different stages. Retrieval needs embeddings and reranking, classification needs issue routing, generation needs strong reasoning, and verification needs deterministic and model-assisted checks.

CHAPTER 3: REQUIREMENTS

3.1 Hardware Requirements

The system can run in a lightweight local mode, but full retrieval and heavy NLP features benefit from stronger hardware.

Minimum requirements:

1. Processor: Intel i5 or equivalent modern CPU.
2. RAM: 8 GB minimum for light mode and small corpus testing.
3. Storage: At least 10 GB free disk space for code, local corpora, model caches, and vector data.
4. Network: Internet access for API calls, model downloads, and optional remote source ingestion.
5. Operating system: Windows, Linux, or macOS. The current development workspace is Windows.

Recommended requirements:

1. Processor: Intel i7, Ryzen 7, or better.
2. RAM: 16 GB to 32 GB for full local indexing, reranking, and large corpus workflows.
3. Storage: 30 GB or more for legal corpora, Hugging Face model caches, and Qdrant storage.
4. GPU: Optional, but useful for local transformer inference, reranker training, and heavy NLP tasks.
5. Docker support: Recommended for running Qdrant locally.

The project includes a graceful-degradation strategy. If heavy models are not enabled or dependencies are unavailable, the pipeline uses regex, heuristic issue detection, lexical support, and fallback retrieval behavior instead of crashing.

3.2 Software Requirements

The core software requirements are:

1. Python 3.10 or later.
2. Streamlit for the dashboard and multi-page web application.
3. Chainlit for conversational legal QA with visible pipeline steps.
4. LangGraph for the shared legal reasoning pipeline.
5. Qdrant for vector storage, with SQLite fallback support.
6. LangChain and LlamaIndex components for document loading and retrieval workflows.
7. Groq SDK for LLM inference.
8. Hugging Face libraries, including transformers, datasets, sentence-transformers, FlagEmbedding, and huggingface-hub.
9. Pydantic for typed schemas and structured results.
10. Docling and PyPDF for document parsing.
11. spaCy, GLiNER, DeBERTa zero-shot classification, and optional NLI models for legal NLP.
12. RAGAS, LegalBench-style scripts, and local evaluation utilities for benchmarking.
13. Plotly and Pandas for UI and data display.
14. Docker Compose for local service orchestration.

Important environment variables include:

1. `GROQ_API_KEY` for Groq-hosted LLM generation.
2. `QDRANT_URL` for vector database access.
3. `HF_TOKEN` for Hugging Face model access where needed.
4. `COURTLISTENER_TOKEN`, `GOVINFO_API_KEY`, `CONGRESS_API_KEY`, and related source-specific keys for remote legal source ingestion.
5. `OMNILEGAL_ENABLE_HEAVY_MODELS` and related feature flags for enabling heavy NLP components.
6. `OMNILEGAL_CONTEXTUAL_PROVIDER` and `OMNILEGAL_CONTEXTUAL_MODEL` for contextual retrieval configuration.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 Overview of the System

OmniLegal AI follows a modular architecture divided into six major layers:

1. User interface layer.
2. Shared legal reasoning pipeline.
3. Retrieval and vector storage layer.
4. Corpus ingestion and source adapter layer.
5. Service workflow layer.
6. Evaluation and training layer.

The user interacts with the system through Streamlit pages or a Chainlit chat interface. The primary legal QA and analysis workload flows through the shared LangGraph pipeline. The pipeline classifies the input, extracts legal entities and issues, retrieves relevant legal authorities, analyzes jurisdictions, synthesizes an answer, and verifies citations.

The system maintains legal data in corpus directories, dataset registries, donor registries, remote source manifests, and vector collections. It supports both preserved legacy collections such as `INTL_TREATIES`, `NATIONAL_IN`, `CASE_LAW`, and `SHAW_PRIVATE`, and granular production collections such as `CASE_LAW_US`, `CASE_LAW_IN`, `STATUTES_EU`, and `COMMENTARY_GLOBAL`.

4.2 Architecture Description

The core architecture can be described as the following pipeline:

User Query -> Classification -> Entity and Issue Extraction -> Hybrid Retrieval -> Jurisdiction Analysis -> Synthesis and Conflict Detection -> Citation Verification -> Final Answer

The Streamlit homepage (`app.py`) acts as the project dashboard. It shows local corpus status, latest evaluation artifacts, Qdrant collection status, donor repositories, dependency status, model cache status, remote source ingestion status, translation status, and production completion gates.

The Chainlit frontend (`chainlit_app.py`) exposes the same pipeline as a conversational chat experience. It shows each step in a collapsible pane: classification, entities and issues, retrieval, jurisdiction analysis, synthesis, citation verification, and final answer. It also supports PDF uploads and ingestion into user-specific collections.

The LangGraph pipeline (`src/pipeline/graph.py`) is the single source of truth for end-to-end legal analysis. It connects the following nodes:

1. `classify_input`
2. `extract_entities`
3. `retrieve`
4. `analyze_jurisdictions`
5. `synthesize`
6. `verify_citations`

This structure ensures that different interfaces can reuse the same backend behavior instead of duplicating logic.

4.3 Components Description

The major components are:

1. Configuration component: `src/config.py` stores directory paths, source files, collection names, model names, API keys, feature flags, routing tables, chunking settings, and ingestion phases.
2. Schema component: `src/schemas.py` defines typed Pydantic models for citations, retrieved passages, entity tags, QA results, council results, conflict results, stance results, briefs, evaluation metrics, argument maps, debate cards, benchmarks, and pipeline states.
3. Data registry component: `src/data/registry.py` manages dataset records for runtime, training, and evaluation usage. The local registry currently contains 17 datasets.
4. Corpus catalog component: `src/data/corpus_catalog.py` normalizes project PDFs and curated legal datasets into shared document records. It supports JSON, JSONL, CSV, XLSX, text, text-directory, and PDF sources.
5. Ingestion component: `src/rag/ingestion.py` parses and chunks treaties, case law, commentary, national law, and remote source material. It includes structure-aware chunking for legal texts.
6. Vector store component: `src/rag/vector_store.py` provides Qdrant and SQLite vector-store abstractions. It manages collection creation, chunk upserts, collection counts, and embedding model loading.
7. Retrieval component: `src/rag/retriever.py` and `src/pipeline/retriever_node.py` implement hybrid search, routing, query normalization, jurisdiction filters, source diversity, reranking, linked passage expansion, and noise filtering.
8. Classification and entity extraction component: `src/pipeline/classifier.py` and `src/pipeline/entity_extractor.py` classify inputs, detect legal issue labels, extract named entities, infer jurisdictions, infer temporal frames, detect named cases, and identify query intent.
9. Jurisdiction analyzer component: `src/pipeline/jurisdiction_analyzer.py` performs jurisdiction-specific IRAC-style reasoning using LLM calls where available and fallback analysis otherwise.
10. Synthesizer component: `src/pipeline/synthesizer.py` performs multi-jurisdiction synthesis, temporal weighting, approximate Shepardizing, conflict detection, and final draft creation.
11. Citation verifier component: `src/pipeline/citation_verifier.py` verifies citation markers, quoted text, lexical support, source mismatch, optional NLI support, self-critique, and grounded fallback answer generation.
12. Service layer: `src/services` contains reusable workflows for QA, brief generation, stance prediction, argument mining, conflict detection, benchmarks, remote source ingestion, translation preparation, production controls, and model cache management.
13. User interface layer: `pages/` contains Streamlit pages for Supreme Omni Model, Corpus Explorer, Legal QA, Legal Analyzer, Model Council, Stance Predictor, Brief Generator, Debate Coach, and Benchmarks.
14. Scripts and CLIs: `src/cli` and `scripts` provide commands for ingestion, source audit, benchmark execution, model prewarming, DSPy tuning, retrieval evaluation, translation preparation, and completion gate verification.

CHAPTER 5: METHODOLOGY

5.1 System Design

The methodology is based on modular legal RAG. Instead of sending a user question directly to an LLM, the system processes the question through multiple deterministic and model-assisted stages.

First, the input is classified to identify whether it is a legal question, policy prompt, document analysis request, or comparison query. Next, entities and issues are extracted using a combination of heuristics, legal aliases, pattern matching, optional spaCy legal NER, GLiNER, and zero-shot classification. The system then builds collection-specific queries and routes them to the appropriate legal corpora.

Retrieved passages are analyzed and converted into jurisdiction-specific reasoning. The synthesizer combines these analyses into a structured answer and detects conflicts between legal regimes where applicable. Before the final answer is shown, the citation verifier checks whether citations are valid and whether claims are sufficiently supported by retrieved passages. This multi-stage design reduces hallucination risk and makes the system easier to debug and evaluate.

The design also separates product workflows from core pipeline logic. For example, Legal QA, Legal Analyzer, Supreme Omni Model, and Chainlit chat all use the shared pipeline, while Brief Generator and Debate Coach reuse service-layer functions that depend on retrieval, stance prediction, and argument mining.

5.2 Model Selection

The model stack was selected based on the needs of legal research:

1. LLM generation requires strong reasoning and summarization. Groq-hosted Llama 3.3 70B is used for high-quality synthesis.
2. Local fallback requires accessible inference. Ollama with Qwen 2.5 7B is configured as a local LLM option.
3. Embedding requires multilingual and legal-domain retrieval capability. BAAI/bge-m3 is selected as the dense embedding backbone.
4. Reranking requires better ordering of retrieved passages. BAAI/bge-reranker-v2-m3 is configured for cross-encoder reranking.
5. Classification requires flexible issue routing. DeBERTa zero-shot classification is configured for legal issue labels.
6. Entity extraction requires both domain-specific and flexible extraction. spaCy legal NER and GLiNER are supported, with regex and alias fallback.
7. Verification requires factual support checking. The system includes lexical support checks and optional NLI verification with Vectara's hallucination evaluation model.
8. Contextual retrieval benefits from document-level summaries. Gemini 2.5 Flash is configured as an optional contextual retrieval provider.

This mixed approach is more practical than relying on one model for every task. Each model is used where its strengths match the pipeline requirement.

5.3 Prompt Engineering and Legal Reasoning Design

Prompt engineering is used in multiple parts of the system. The Supreme Omni Model prompt instructs the LLM to synthesize results from the model council, QA agent, stance predictor, and debate coach into a single verdict with an executive summary, legal reasoning, and diplomatic recommendation. It also instructs the model to use only provided authorities and cite them in `[Source Name, Page]` format.

The jurisdiction analysis prompt follows an IRAC-style pattern: issue, rule, application, and conclusion. This format is suitable for legal reasoning because it separates legal authority from factual application. It also makes outputs easier to verify.

The retrieval and synthesis components are designed around strict grounding. The pipeline does not rely only on free-form generated answers. It tracks retrieved passages, citations, jurisdiction labels, source names, pages, confidence values, and verification grades.

The citation verifier functions as a prompt-safety and grounding layer. If citations are not supported, incorrect markers can be stripped or the answer can fall back to an evidence-limited response. This is important because legal AI must avoid presenting unsupported statements as authoritative.

5.4 Response Generation Process

The response generation process consists of the following steps:

1. The user enters a legal question or uploads a PDF through Streamlit or Chainlit.
2. The system initializes a `PipelineStateDict` containing the raw input.
3. The classifier assigns an input class and confidence score.
4. The entity extractor identifies legal entities, issues, jurisdictions, named cases, ISO country codes, temporal frames, and query intent.
5. The retriever maps the query to relevant collections such as international treaties, Indian constitutional law, US case law, EU law, commentary, or granular production collections.
6. Hybrid retrieval retrieves candidate passages from Qdrant or fallback storage.
7. The jurisdiction analyzer generates jurisdiction-specific legal analyses.
8. The synthesizer creates a draft answer, applies temporal reasoning, identifies legal conflicts, and formats the response.
9. The citation verifier checks citation markers, quote support, lexical support, and optional NLI support.
10. The final answer is returned with citations, retrieved sources, conflict notes, and runtime warnings if any stage degraded.

For MUN-specific workflows, the same information is reused to generate stance predictions, one-page briefs, debate cards, rebuttals, likely POIs, and negotiation red lines.

5.5 Evaluation Process

The evaluation methodology combines several metric families:

1. Retrieval metrics: recall at 5, recall at 10, citation coverage, and source diversity.
2. Citation metrics: citation existence rate, quote match, citation correctness, and unsupported claim rate.
3. Legal task metrics: macro-F1 for conflict detection and stance-style labels.
4. Argument metrics: average argument spans and cited span rate.
5. RAG quality metrics: RAGAS faithfulness and related production gates.
6. Benchmark metrics: LegalBench-style runs and stratified legal smoke queries.
7. Generation overlap metrics: ROUGE-L and BERTScore are configured in experiment defaults. BLEU can be added for college-report consistency, but it should be interpreted carefully because legal answers often use valid paraphrases rather than exact wording.

The evaluation process uses local JSON and JSONL artifacts stored under `data/evaluation`, `data/evals`, and `data/gold`. Smoke benchmarks are used to validate the system quickly, while larger benchmark datasets remain

registry-backed and can be run separately.

CHAPTER 6: IMPLEMENTATION AND DEPLOYMENT

6.1 Implementation Overview

OmniLegal AI is implemented as a Python application with a modular backend and multiple frontends. The root Streamlit app (`app.py`) provides a dashboard for system status, while individual pages under `pages/` provide user-facing workflows. Chainlit (`chainlit_app.py`) provides a chat-based interface with visible pipeline stages.

The implementation emphasizes reusable service layers. Core legal QA does not live directly inside UI files. Instead, the UI calls the shared LangGraph pipeline or service functions. This improves maintainability because changes to retrieval, citation verification, or synthesis automatically affect all frontends that use the shared pipeline.

The project also includes command-line utilities for ingestion, evaluation, training preparation, model prewarming, source auditing, and production readiness checks. This makes the system suitable for both interactive use and repeatable experiments.

6.2 Tools and Technologies Used

The main tools and technologies used are:

1. Python: Core programming language.
2. Streamlit: Multi-page web application interface.
3. Chainlit: Conversational chat interface with step-by-step pipeline visualization.
4. LangGraph: State-machine orchestration for the legal reasoning pipeline.
5. Qdrant: Vector database for semantic search.
6. SQLite: Local fallback vector store and cache support.
7. BAAI/bge-m3: Embedding model.
8. BAAI/bge-reranker-v2-m3: Reranking model.
9. Groq SDK: LLM inference using Llama 3.3 70B.
10. Hugging Face transformers and datasets: Model loading and benchmark support.
11. Docling and PyPDF: PDF/document parsing.
12. Pydantic: Structured typed outputs.
13. RAGAS: RAG quality evaluation.
14. DSPy: Optional prompt/program optimization for jurisdiction analysis.
15. Docker Compose: Local Qdrant service management.
16. NetworkX: Citation graph and related structural processing.
17. Plotly and Pandas: Data display in the UI.

6.3 Code Implementation Details

The implementation is organized as follows:

1. `app.py`: Main Streamlit dashboard showing project overview, corpus counts, evaluation artifacts, dependency status, model cache status, remote ingestion status, translation status, and production completion gates.
2. `chainlit_app.py`: Chainlit frontend that runs the full pipeline and shows each step as a collapsible tool pane. It includes PDF upload ingestion support.

3. `src/config.py`: Central configuration file for directories, corpus files, vector database settings, collection names, model identifiers, API keys, feature flags, routing maps, and chunking settings.
4. `src/schemas.py`: Pydantic schema definitions for citations, retrieved passages, QA results, council submissions, conflict results, stance predictions, briefs, evaluation artifacts, argument maps, debate cards, and pipeline states.
5. `src/pipeline/graph.py`: LangGraph state machine connecting classification, extraction, retrieval, jurisdiction analysis, synthesis, and citation verification.
6. `src/pipeline/classifier.py`: Input classification using regex heuristics and optional zero-shot classification.
7. `src/pipeline/entity_extractor.py`: Legal entity extraction using spaCy, GLiNER, aliases, fuzzy matching, concept detection, issue classification, jurisdiction inference, and temporal inference.
8. `src/pipeline/retriever_node.py`: Intent-first retrieval with collection filtering, jurisdiction filtering, source diversity, named-case handling, query variants, and confidence scoring.
9. `src/pipeline/jurisdiction_analyzer.py`: Jurisdiction-specific IRAC analysis using LLM calls with fallback behavior.
10. `src/pipeline/synthesizer.py`: Multi-jurisdiction synthesis, conflict detection, temporal weighting, case seeding, and legal draft generation.
11. `src/pipeline/citation_verifier.py`: Citation marker verification, quote matching, lexical support checks, NLI support, self-critique, incorrect-marker stripping, and fallback answer construction.
12. `src/rag/ingestion.py`: Structure-aware legal corpus ingestion for treaties, commentary, national law, case law, and remote sources.
13. `src/rag/retriever.py`: Hybrid search, dense embedding, sparse retrieval, RRF merge, reranking, fallback search, and collection routing.
14. `src/rag/vector_store.py`: Qdrant and SQLite vector-store abstraction.
15. `src/services/retrieval_qa.py`: Reusable legal QA service that builds context, deduplicates citations, and returns structured answers.
16. `src/services/conflict_detection.py`: International-versus-domestic legal relationship analysis.
17. `src/services/stance_prediction.py`: India stance prediction from retrieved authorities and conflict analysis.
18. `src/services/brief_generation.py`: One-page MUN brief generation with citations.
19. `src/services/argument_mining.py`: Argument span extraction and debate card generation.
20. `src/services/benchmarks.py`: Local smoke benchmarks for QA, conflict detection, stance prediction, brief generation, and argument mining.
21. `src/services/remote_sources.py`: Remote legal source cataloging and bounded ingestion.
22. `src/services/adapters/`: Source adapters for CourtListener, GovInfo, EUR-Lex/CELLAR, CD-ICJ, Indian Supreme Court AWS data, UK Find Case Law, UN Digital Library, RusLawOD, and Israel Versa.
23. `src/cli/`: Command-line utilities for evaluation, ingestion, source audit, retrieval eval, DSPy tuning, model prewarming, translation preparation, and production gate verification.
24. `pages/`: Streamlit workflows for Supreme Omni Model, Corpus Explorer, Legal QA, Legal Analyzer, Model Council, Stance Predictor, Brief Generator, Debate Coach, and Benchmarks.

6.4 Deployment Process

The application can be deployed locally or on a suitable web-hosting platform that supports Python applications and environment variables.

Local deployment steps:

1. Create and activate a Python virtual environment.
2. Install dependencies using `pip install -r requirements.txt`.
3. Configure required environment variables in `.env`, especially `GROQ_API_KEY` and `QDRANT_URL`.
4. Start Qdrant using Docker Compose if using the Qdrant backend.
5. Ensure collections are created using the collection utility.
6. Ingest local corpora or remote sources using the ingestion CLIs.
7. Run the Streamlit dashboard with `streamlit run app.py`.
8. Run the Chainlit chat interface with `chainlit run chainlit_app.py`.
9. Execute benchmark or readiness commands such as `python -m src.cli.benchmarks`, `python -m src.cli.doctor`, and `python -m src.cli.verify_completion_gates`.

For cloud deployment, the system should use managed secret storage for API keys and should avoid bundling private legal corpora unless redistribution rights are confirmed. The application should also expose legal disclaimers, rate limiting, logs, and source verification warnings.

6.5 Application Interface

The Streamlit interface contains the following pages:

1. Supreme Omni Model: Orchestrates model council, grounded QA, India stance prediction, debate preparation, and final Supreme Verdict synthesis.
2. Corpus Explorer: Allows inspection of indexed legal collections and corpus metadata.
3. Legal QA: Provides citation-grounded answers to international, Indian, and comparative legal questions.
4. Legal Analyzer: Performs entity intake and international-versus-Indian conflict discovery.
5. Model Council: Runs multi-model deliberation for hard questions.
6. Stance Predictor: Predicts India's likely legal or diplomatic stance for a given issue.
7. Brief Generator: Produces a fixed-format one-page MUN brief with citations and stance analysis.
8. Debate Coach: Generates opening arguments, likely POIs, rebuttal lines, negotiation red lines, and argument maps.
9. Benchmarks: Displays benchmark coverage and runs local smoke evaluations.

The Chainlit interface provides a chat-first experience. It is especially useful for explaining the pipeline because each backend stage is displayed separately, allowing users to see classification, entity extraction, retrieval, jurisdiction analysis, synthesis, citation verification, and final answer generation.

CHAPTER 7: EXPERIMENTAL RESULTS

7.1 Evaluation Setup

The experimental setup uses local evaluation artifacts already present in the project. These artifacts are stored under `data/evaluation`, `data/evals`, and `data/gold`. They test retrieval, QA, conflict detection, argument mining, legal smoke queries, RAGAS faithfulness, ingestion quality, translation readiness, and production completion gates.

The evaluation data includes:

1. `retrieval_smoke.json`: Tests retrieval recall, citation coverage, and source diversity.
2. `qa_smoke.json`: Tests answer-with-citation rate and comparative query detection.
3. `conflict_smoke.json`: Tests conflict detection against a small manual gold file.
4. `argument_smoke.json`: Tests generated argument span coverage and citation support.
5. `stratified_queries.jsonl`: Contains legal queries grouped by legal area, including use of force, IHL, human rights, treaty interpretation, state responsibility, and jurisdiction/immunity.
6. `latest_completion_gates.json`: Records production-readiness gate results.
7. `latest_ingestion_quality.json`: Samples vector collections and checks metadata quality.
8. LegalBench and RAGAS result artifacts under `data/evals/results`.

The evaluation is not limited to text overlap. It focuses on legal reliability, citation grounding, retrieval quality, and production readiness.

7.2 Results and Observations

The retrieval smoke benchmark reports three records with the following results:

1. Recall at 5: 0.3333.
2. Recall at 10: 0.3333.
3. Citation coverage: 1.0000.
4. Source diversity: 1.0000.

These results show that the system returns citations and diverse sources, but retrieval recall needs improvement for stronger source matching.

The QA smoke benchmark reports:

1. Answer with citation rate: 1.0000.
2. Comparative query detection rate: 0.5000.

This indicates that the QA layer reliably attaches citations in smoke tests, while comparative query detection should be improved.

The conflict detection smoke benchmark reports:

1. Macro-F1: 1.0000.

This is a positive result, but the gold file is very small and should be expanded before making strong claims.

The argument mining smoke benchmark reports:

1. Average argument spans: 8.0000.
2. Cited span rate: 1.0000.

This shows that the Debate Coach can generate citation-backed argument spans in the smoke setup.

The latest legal smoke run reports:

1. Total queries: 54.
2. Errors: 0.
3. Hallucination rate: 0.0000.
4. Citation existence rate: 1.0000.
5. Quote match: 1.0000.
6. Total citations: 204.
7. Correct citations: 204.

This demonstrates strong smoke-test performance for citation existence and quote matching. However, production completion gates still report the overall status as `not_ready`, mainly due to failed or incomplete RAGAS faithfulness and older citation gate artifacts.

The completion gate artifact reports:

1. Remote checkpoint: pass.
2. Required collections nonempty: pass.
3. LegalBench run: pass.
4. Translation strategy recorded: pass.
5. RAGAS faithfulness: fail.
6. Citation existence gate: fail in the referenced older artifact due to null values.
7. Quote-match gate: fail in the referenced older artifact due to null values.
8. Unsupported rate: pass.

The ingestion quality artifact reports a failure because sampled chunks are missing metadata fields such as document hash, canonical document ID, legal type, and importance score. This is important because production-grade legal retrieval needs rich metadata for filtering, reproducibility, and citation verification.

7.3 Performance Analysis

The results show that OmniLegal AI is functionally strong as a local legal research prototype. It can answer legal questions, attach citations, run a multi-step pipeline, support MUN workflows, and generate benchmark artifacts. The legal smoke run is particularly strong, with zero errors across 54 queries and perfect citation existence and quote-match rates in that artifact.

At the same time, the evaluation artifacts show that the system should not yet be described as fully production-ready. Retrieval recall in the small retrieval smoke test is low, comparative query detection is only partially successful, RAGAS faithfulness has not passed, and ingestion metadata quality requires improvement. The conflict and argument benchmarks are promising but based on small seed datasets.

Therefore, the correct interpretation is that OmniLegal AI demonstrates a complete architecture and strong smoke-test behavior, but needs broader datasets, stronger retrieval evaluation, cleaner metadata, and more robust production gates before legal deployment.

CHAPTER 8: DISCUSSION

8.1 Interpretation of Results

The experimental results confirm that a modular legal RAG system can reduce some common weaknesses of general-purpose LLM legal assistants. Citation existence and quote matching are strong in the latest smoke run, suggesting that the verifier and retrieval layer are effective in controlled tests. The system also demonstrates practical workflows beyond ordinary QA, including brief generation, debate coaching, stance prediction, and conflict discovery.

The results also highlight the importance of evaluation design. ROUGE and BLEU are useful for measuring text overlap, but legal AI needs additional metrics. A legally valid answer may use different wording from a reference answer, so overlap metrics can understate quality. Conversely, a fluent answer may have high overlap but poor citation grounding. OmniLegal therefore evaluates citation existence, quote match, retrieval recall, source diversity, faithfulness, and unsupported claim rate.

The production gate output is especially useful because it identifies what still needs improvement. The system already has nonempty required collections and remote ingestion checkpoints, but RAGAS faithfulness and ingestion metadata quality must be improved. This makes the project report stronger because it can honestly discuss both working features and current limitations.

8.2 Comparison with Expected Output

The expected output of the system is a legally structured, citation-grounded answer that includes relevant authorities and avoids unsupported claims. Compared with this expectation, the current implementation performs well in smoke tests. It routes questions through a multi-stage pipeline, retrieves passages, generates jurisdiction-specific reasoning, verifies citations, and returns a final answer.

For MUN use cases, the expected output is not only a legal answer but also a practical debate package. The Supreme Omni Model combines council deliberation, QA, Indian stance prediction, and debate preparation. The Brief Generator produces fixed-format sections, while the Debate Coach produces opening points, POIs, rebuttals, and negotiation red lines. This meets the broader project goal of legal research assistance for student and MUN workflows.

However, compared with a production legal research tool, the system still requires improvement. It needs larger benchmark datasets, stronger human-reviewed references, better multilingual handling, cleaner metadata, more complete remote-source credentials, and formal legal validation. Therefore, the current output is best described as a strong academic prototype and evaluation framework rather than a professional legal advice product.

CHAPTER 9: ADVANTAGES AND LIMITATIONS

9.1 Advantages

OmniLegal AI provides several advantages:

1. Domain-specific legal focus: The system is built specifically for legal research instead of generic text generation.
2. Source-grounded answers: Responses are based on retrieved legal passages and include citations.
3. Multi-jurisdiction support: The system supports international law, Indian law, US law, UK law, EU law, Russian law, Israeli law, and commentary collections.
4. Shared pipeline architecture: Streamlit and Chainlit interfaces reuse the same LangGraph pipeline.
5. Citation verification: The verifier checks citation markers, quote support, source mismatch, and lexical support.
6. MUN-oriented workflows: The project supports stance prediction, brief generation, debate cards, POIs, rebuttals, and diplomatic recommendations.
7. Modular source ingestion: Remote adapters support CourtListener, GovInfo, EUR-Lex, CD-ICJ, UN Digital Library, Indian Supreme Court AWS data, UK Find Case Law, RusLawOD, and Israel Versa.
8. Evaluation-aware design: Benchmarks, smoke tests, RAGAS artifacts, LegalBench runs, completion gates, and ingestion quality checks are included.
9. Graceful degradation: Heavy NLP components are optional, and the system can fall back to heuristic behavior.
10. Typed outputs: Pydantic schemas make results structured, reusable, and easier to validate.
11. Local and remote flexibility: The system supports local corpora, remote source ingestion, Qdrant vector storage, SQLite fallback, cloud APIs, and local model options.

9.2 Limitations

The system also has limitations:

1. It is not a substitute for a qualified lawyer. Outputs must be verified directly against legal sources.
2. It depends on external API keys for the highest-quality LLM generation and some remote source ingestion.
3. Some source adapters require credentials, permissions, or licensing confirmation.
4. Retrieval recall is still limited in the smoke benchmark.
5. Comparative query detection requires improvement.
6. RAGAS faithfulness gates currently show production-readiness failure.
7. Ingestion quality checks show missing metadata in sampled chunks.
8. Manual gold datasets for conflict, stance, and brief evaluation are currently small.
9. Heavy NLP components require additional memory, disk space, and model downloads.
10. Multilingual support is present in configuration and collection design, but translation preparation currently records no provider when translation keys are unavailable.
11. The system may return incomplete answers when collections are empty, Qdrant is unavailable, or source ingestion has not been run.

12. Some report-level metrics such as BLEU are not currently implemented directly and should be added if required by the evaluation format.

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

OmniLegal AI successfully demonstrates the design and implementation of a domain-specific legal research assistant using hybrid retrieval, multi-model synthesis, and citation evaluation. The system is more than a basic summarizer because it includes a shared LangGraph pipeline, multiple user interfaces, legal corpus ingestion, vector storage, retrieval routing, legal entity extraction, jurisdiction analysis, synthesis, citation verification, model council deliberation, stance prediction, brief generation, debate coaching, and benchmark support.

The project shows that legal AI systems should be built around source grounding and evaluation rather than free-form generation alone. Existing smoke-test artifacts show strong citation existence and quote matching, while production-readiness artifacts identify important areas for improvement. This balanced result makes OmniLegal suitable as an academic AI project because it has a working prototype, measurable outputs, and clear future work.

Overall, OmniLegal AI demonstrates how retrieval-augmented generation and structured verification can improve legal research workflows for students, researchers, and MUN delegates. It also shows that responsible legal AI must include disclaimers, citation checks, evaluation metrics, and transparent limitations.

10.2 Future Scope

The future scope of OmniLegal AI includes:

1. Expanding the gold evaluation datasets for conflict detection, stance prediction, brief generation, QA, and retrieval.
2. Adding BLEU and ROUGE-L evaluation for generated legal summaries and briefs if required by academic reporting.
3. Improving RAGAS faithfulness by refining prompts, retrieval context selection, and citation verification.
4. Fixing ingestion metadata quality by adding document hashes, canonical document IDs, legal type, and importance scores to all chunks.
5. Training or fine-tuning the reranker using generated retrieval triples.
6. Adding stronger multilingual support for Russian, Hebrew, EU multilingual law, and Indian regional-language legal materials.
7. Implementing authenticated user sessions and per-user workspaces for uploaded legal documents.
8. Adding source-level licensing and redistribution checks before public deployment.
9. Integrating more official legal sources and court APIs.
10. Improving the Corpus Explorer with visual dashboards for collection health and legal coverage.
11. Adding human-in-the-loop review workflows for legal experts to approve or reject generated answers.
12. Deploying the system on a managed cloud platform with monitored Qdrant, API secret management, rate limiting, and logging.
13. Creating a PDF and DOCX export feature for generated briefs and debate cards.
14. Adding a formal uncertainty score that combines retrieval confidence, citation support, and model agreement.
15. Extending the Supreme Omni Model to compare multiple national positions for MUN simulations.

CHAPTER 11: REFERENCES

1. Streamlit Official Documentation.
2. Chainlit Official Documentation.
3. LangGraph Documentation.
4. Qdrant Vector Database Documentation.
5. Groq API Documentation.
6. Hugging Face Transformers Documentation.
7. BAAI bge-m3 and bge-reranker model documentation.
8. RAGAS Documentation for Retrieval-Augmented Generation Evaluation.
9. LegalBench benchmark resources.
10. LexGLUE benchmark resources.
11. CaseHOLD legal QA benchmark resources.
12. CLERC legal retrieval benchmark resources.
13. LLeQA legal question answering benchmark resources.
14. BSARD statutory retrieval benchmark resources.
15. UN Charter, ICCPR, ICESCR, and Indian Constitution source documents used in the local corpus.
16. Malcolm N. Shaw, International Law, used as legal commentary in the local corpus.
17. CourtListener, GovInfo, EUR-Lex, UN Digital Library, CD-ICJ, UK Find Case Law, RusLawOD, Indian Supreme Court AWS Open Data, and Israel Versa source adapters referenced by the codebase.

APPENDIX A: REPORT SECTION FILLING CHECKLIST

Use this checklist to match the attached medical-report format while converting it into the OmniLegal version.

Cover page:

1. Replace the title with "OmniLegal AI: Domain-Specific Legal Research Assistant Using Hybrid RAG, Multi-Model Synthesis and Citation Evaluation".
2. Add student names and register numbers.
3. Keep degree, department, institution, university, and month/year as required by the college.

Bonafide certificate:

1. Replace the medical-report title with the OmniLegal title.
2. Replace student names and register numbers.
3. Replace guide name, designation, and viva date.

Declaration:

1. Replace the project title.
2. Replace guide details.
3. Add student signatures and register numbers.

Acknowledgement:

1. Keep the standard college acknowledgement style.
2. Add guide and coordinator names.
3. Mention help received during implementation, testing, and report preparation.

Abstract:

1. Explain the legal research problem.
2. Mention hybrid RAG, legal corpora, LangGraph, Qdrant, Streamlit, Chainlit, LLM synthesis, and citation evaluation.
3. Mention benchmark metrics and current results.

Chapter 1:

1. Background: Explain AI in legal research and why citation-grounded answers matter.
2. Motivation: Explain MUN, legal research, Indian law, international law, and the need for structured outputs.
3. Problem statement: Explain hallucination, manual research effort, and source verification.
4. Objectives: List retrieval, entity extraction, analysis, citation verification, UI, and evaluation goals.

Chapter 2:

1. Existing systems: Legal databases, legal NLP datasets, general LLMs, and RAG systems.
2. Limitations: Subscription access, hallucinations, weak citation verification, lack of end-to-end evaluation.
3. Research gap: Need for accessible academic legal RAG with MUN outputs and evaluation.
4. Base model selection: Mention Groq Llama 3.3 70B, BAAI/bge-m3, bge-reranker, DeBERTa, GLiNER, spaCy, Gemini contextual retrieval, and local Qwen fallback.

Chapter 3:

1. Hardware: CPU, RAM, storage, GPU optional, Docker support.
2. Software: Python, Streamlit, Chainlit, LangGraph, Qdrant, Groq, Hugging Face, Docling, PyPDF, Pydantic, RAGAS, Docker.

Chapter 4:

1. Explain six layers: UI, pipeline, retrieval, ingestion, service workflows, evaluation.
2. Include pipeline flow: input -> classify -> extract -> retrieve -> analyze -> synthesize -> verify -> final.
3. Add component descriptions mapped to actual code modules.

Chapter 5:

1. Explain modular legal RAG methodology.
2. Explain model selection by task.
3. Explain prompt engineering and IRAC reasoning.
4. Explain response generation process.
5. Explain evaluation metrics, including citation metrics, retrieval recall, RAGAS, LegalBench, macro-F1, ROUGE-L, and optional BLEU.

Chapter 6:

1. Explain implementation with actual files and folders.
2. Add tools and technologies.
3. Add deployment commands for Streamlit and Chainlit.
4. Explain application pages.

Chapter 7:

1. Add evaluation setup from existing JSON artifacts.
2. Add results: 54 smoke queries, 0 errors, 0 hallucination rate, 1.0 citation existence, 1.0 quote match, 204 citations, 204 correct citations.
3. Add retrieval smoke results, QA smoke results, conflict smoke results, argument smoke results.
4. Mention production gate status is currently not ready and explain why.

Chapter 8:

1. Interpret strengths and weaknesses.
2. Compare expected citation-grounded legal answers with actual smoke-test results.

Chapter 9:

1. Add advantages: domain focus, citations, multi-jurisdiction support, MUN workflows, evaluation, modularity.
2. Add limitations: legal disclaimer, API dependency, metadata gaps, small gold datasets, incomplete production gates, translation provider gaps.

Chapter 10:

1. Summarize successful implementation.
2. Add future scope: larger datasets, better metadata, fine-tuned reranker, BLEU/ROUGE evaluation, multilingual translation, expert review, deployment, exports.

References:

1. Add documentation references for Streamlit, Chainlit, LangGraph, Qdrant, Groq, Hugging Face, RAGAS, LegalBench, LexGLUE, and legal source documents.

APPENDIX B: CODEBASE MODULE MAP

Root files:

1. `app.py`: Streamlit dashboard and admin overview.
2. `chainlit_app.py`: Chainlit legal chat pipeline.
3. `requirements.txt`: Python dependency list.
4. `docker-compose.yml`: Local service orchestration, mainly Qdrant.
5. `model_capabilities.yaml`: Model selection guidance.
6. `configs/experiment_defaults.json`: Experiment defaults and evaluation metric plan.
7. `configs/landmark_registry.yaml`: Landmark case aliases, tags, and citation graph policy.

Important source folders:

1. `src/pipeline`: End-to-end legal reasoning graph.
2. `src/rag`: Ingestion, retrieval, generation, contextual retrieval, and vector store.
3. `src/services`: Reusable legal workflows and production controls.
4. `src/services/adapters`: Remote legal source adapters.
5. `src/data`: Dataset registry and corpus catalog.
6. `src/models`: Model council, DSPy modules, entailment, heavy NLP loading, and legal NER.
7. `src/cli`: Evaluation, ingestion, audit, training preparation, and readiness commands.
8. `src/training`: Reranker and generation training scaffolds.
9. `pages`: Streamlit pages for each user-facing workflow.
10. `tests`: Unit and integration tests for pipeline behavior, source adapters, corpus catalog, production completion, and retrieval control.

Key collections:

1. `INTL_TREATIES`
2. `NATIONAL_IN`
3. `NATIONAL_US`
4. `NATIONAL_UK`
5. `NATIONAL_EU`
6. `NATIONAL_RU`
7. `NATIONAL_IL`
8. `CASE_LAW`
9. `SHAW_PRIVATE`
10. `COMMENTARY`
11. `CASE_LAW_GLOBAL`
12. `CASE_LAW_US`
13. `CASE_LAW_IN`
14. `CASE_LAW_EU`

15. `CASE_LAW_UK`
16. `CASE_LAW_RU`
17. `CASE_LAW_IL`
18. `STATUTES_US`
19. `STATUTES_IN`
20. `STATUTES_EU`
21. `STATUTES_UK`
22. `STATUTES_RU`
23. `STATUTES_IL`
24. `COMMENTARY_GLOBAL`

Registered dataset count:

The local dataset registry contains 17 datasets covering runtime, training, evaluation, retrieval, QA, stance prediction, brief generation, argument mining, conflict detection, summarization, and benchmark usage.

Current result summary:

The latest smoke artifact reports 54 queries, 0 errors, 0.0 hallucination rate, 1.0 citation existence rate, 1.0 quote match, 204 total citations, and 204 correct citations. Production gates still report `not_ready`, so the report should present the project as an evaluated academic prototype rather than a completed professional legal product.